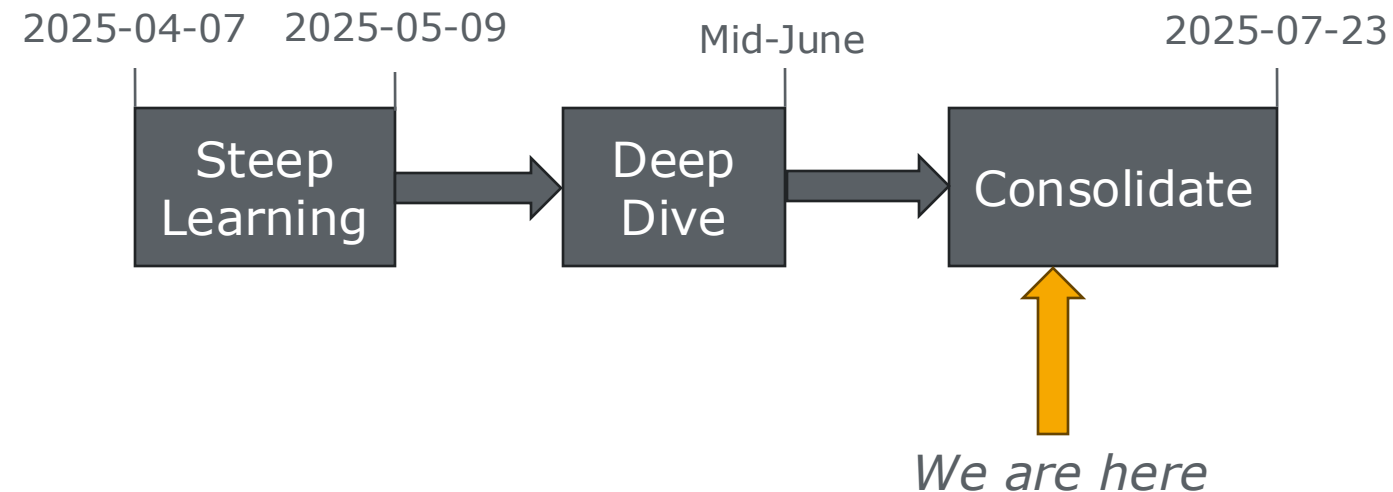




OpenSmalltalk VM vs TruffleSqueak

Enhancing VM Performance: Reviewing and Improving Tools for Code Artifacts



Master Project 2025

Marius Dörbandt, **Oliver Heß**, **Antony Kamp**, **Christoph Thiede**

Advisors: Marcel Taeumel, Jens Lincke, Robert Hirschfeld

Recap: Stacks



OSVM

TSVM



Squeak Image

Interpreter

JIT

GC

(Slang)

TruffleSqueak (Java)

GraalPy

...

Platform Code
(C)

Clang

Simulator
(Smalltalk)

Squeak

Truffle (Java)

Graal Compiler

Hotspot Java VM

Native
Image

OS

Cross-Language Compiler Benchmarking

Are We Fast Yet?

Stefan Marr Benoit Daloze Hanspeter Mössenböck
Johannes Kepler University Linz, Austria
{stefan.marr, benoit.daloze, hanspeter.moessenboeck}@jku.at

Abstract

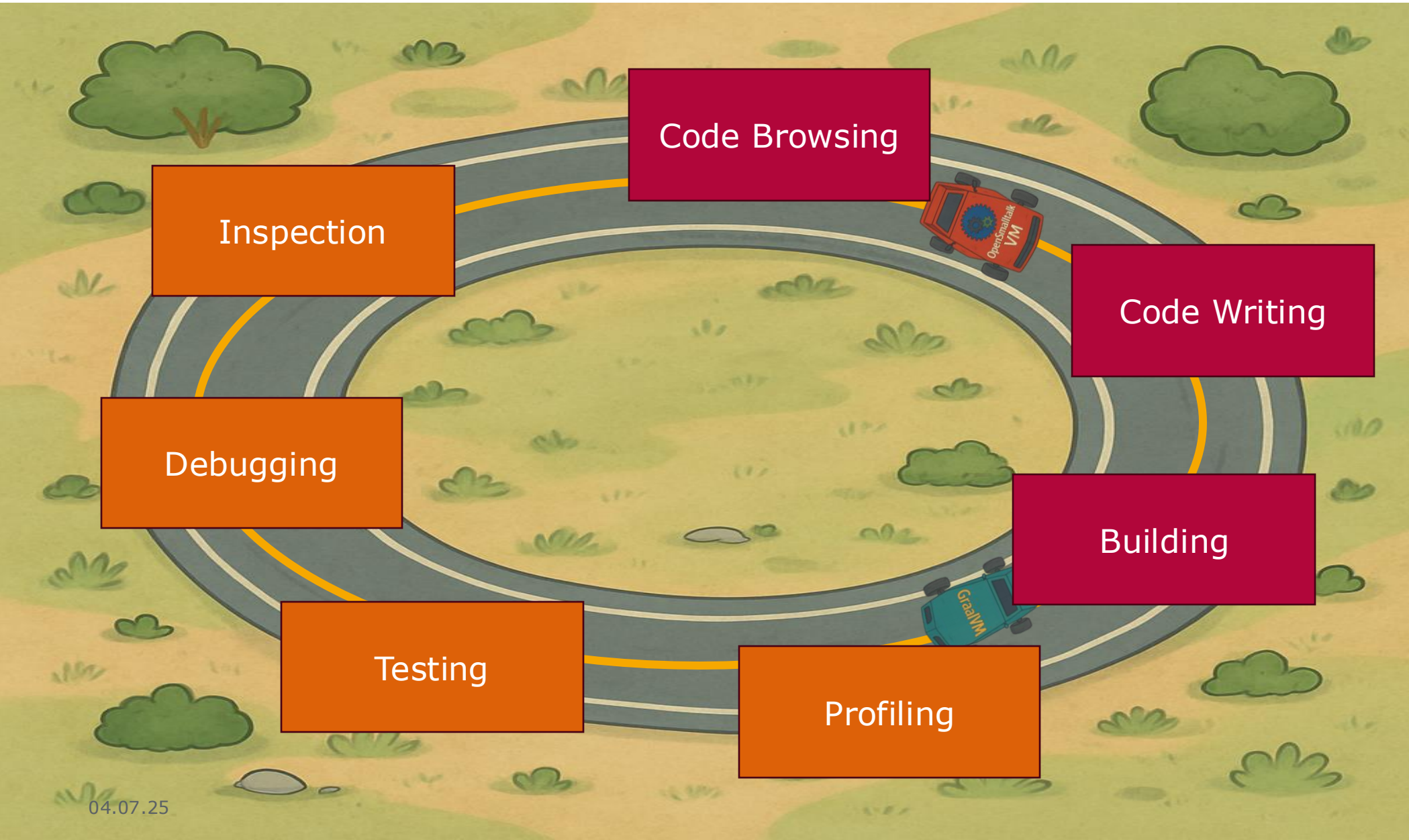
Comparing the performance of programming languages is difficult because they differ in many aspects including preferred programming abstractions, available frameworks, and their runtime systems. Nonetheless, the question about relative performance comes up repeatedly in the research community, industry, and wider audience of enthusiasts.

This paper presents 14 benchmarks and a novel methodology to assess the compiler effectiveness across language implementations. Using a set of common language abstractions, the benchmarks are implemented in Java, JavaScript, Ruby, Crystal, Newspeak, and Smalltalk. We show that the benchmarks exhibit a wide range of characteristics using language-agnostic metrics. Using four different languages on top of the same compiler, we show that the benchmarks perform similarly and therefore allow for a comparison of compiler effectiveness across languages. Based on anecdotal

With the wide variety of languages, there is no trivial answer. It is also unclear whether any single answer is useful outside of a specific context. Nonetheless, the question is asked again and again, which leads us to search for a way to answer one specific question: *is one implementation of a widely used set of core language abstractions as fast as other implementations of the same set of abstractions?*

We conjecture that a commonly supported set of core language abstractions (incl. objects, polymorphic methods, closures, and arrays) is relevant for general application performance. Hence, it is necessary to optimize these abstractions to reach optimal performance. However, the set of abstractions is not universal. For example, programs in Haskell use language abstractions that are different from those in imperative languages. Another aspect is that optimizing only a set of core abstractions is not sufficient for optimal performance of all uses of a language. Programs typically use very ex-

Introduction: VM Development Activities



Code Browsing: Workflow and Discussion

Code Browsing

- **Goals:** Explore behavior, look up implementors/senders/...
 - **Tools:**
 - Both: Squeak's system browser, message traces, ...
 - OSVM: VS Code
 - TSVM: IntelliJ IDEA
 - **Discussion:**
 - **Tabs**/panes vs **windows**, **files** vs **methods**
 - **Static analysis** vs **runtime** debugging/exploration
- 💡 TSVM: Add **static type analysis** to **specialization guards** (IntelliJ Plugin?)
- 💡 OSVM: **Type flow analysis** or search heuristics¹

Code Browsing: Workflow and Discussion

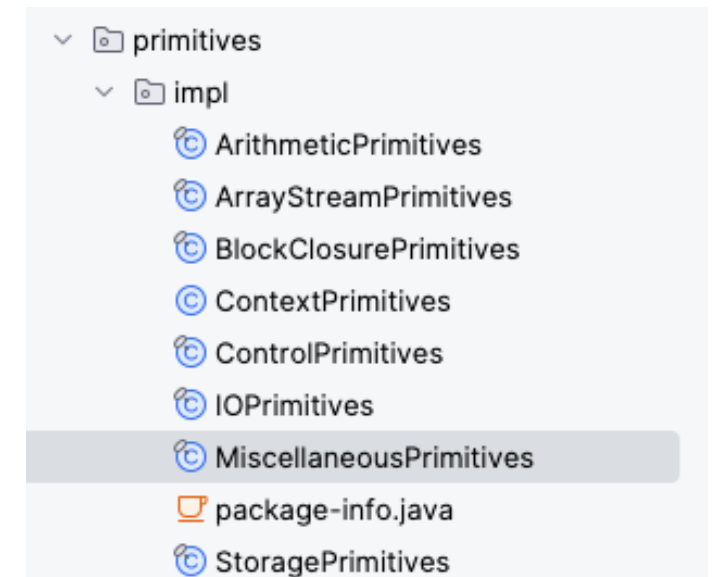
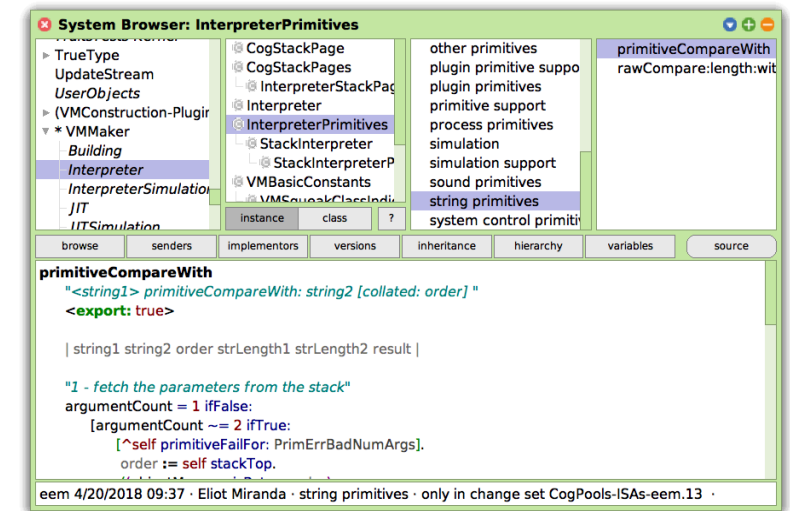


```
@GenerateNodeFactory
@SqueakPrimitive(names = "primitiveCompareString")
public abstract static class PrimCompareStringNode extends AbstractPrimCompareStringNode implements Primitive3 {
    @Specialization(guards = {"string1.isByteType()", "string2.isByteType()", "orderValue == cachedAsciiOrder"}, limit = "1")
    protected static final long doCompareAsciiOrder(@SuppressWarnings("unused") final Object receiver,
        final NativeObject string1, final NativeObject string2, @SuppressWarnings("unused") final NativeObject orderValue,
        @SuppressWarnings("unused") @Cached("asciiOrderOrNull(orderValue)") final NativeObject cachedAsciiOrder) {
        return compareAsciiOrder(string1, string2) + 2L;
    }
    ...
}
```

Code Browsing: Workflow



- **Goals:** Explore behavior, look up implementors/senders/...
- **Used tools**
 - To browse **VM code:**
 - **OSVM:** Squeak (VMMaker), VS Code (platform-specific code, mvm build config)
 - **TSVM:** IntelliJ IDEA
 - To browse **image code:** Squeak
 - System browser, message traces, code search, protocol browser, ...
 - **OSVM:** proxy browser (new tool; more on that later)
 - To understand code: **LLMs** for explaining assembler code / specialization code



- Implementors of primitiveCompareWith

InterpeterPrimitives primitiveCompareWith {string primitives} {VMMaker-Interpreter}

StackInterpeterSimulator primitiveFailFor: {debugging traps} {VMMaker-InterpreterSimulation}

SpurMemoryManager primitiveFailFor: {simulation only} {VMMaker-SpurMemoryManager}

NewObjectMemory primitiveFailFor: {simulation only} {VMMaker-V3MemoryManager}

InterpreterProxy primitiveFailFor: {other} {VMMaker-InterpreterSimulation}

InterpeterPrimitives primitiveFailFor: {primitive support} {VMMaker-Interpreter}

browse

senders

implementors

versions

inheritance

hierarchy

variables

source

primitiveCompareWith

```

"<string1> primitiveCompareWith: string2 [collated: order]"
<export: true>

| string1 string2 order strLength1 strLength2 result |

"1 - fetch the parameters from the stack"
argumentCount = 1 ifFalse:
    [argumentCount ~ 2 ifTrue:
        [^self primitiveFailFor: PrimErrBadNumArgs].
        order := self stackTop.
        ((objectMemory isBytes: order)
            and: [(objectMemory numBytesOfBytes: order) = 256]) ifFalse:
            [^self primitiveFailFor: PrimErrBadArgument]].
    
```

eem 4/20/2018 09:37 · Eliot Miranda · string primitives · only in change set CogPools-ISAs-eem.13 ·

```
@GenerateNodeFactory 32 usages 1 inheritor  ⚡ Fabio Niephaus +1*
@SqueakPrimitive(names = "primitiveCompareString")
public abstract static class PrimCompareStringNode extends AbstractPrimCompareStringNode implements Primitive3 {

    @Specialization(guards = {"string1.isTruffleStringType()", "string2.isTruffleStringType()", "orderValue == cachedAsciiOrder"}, limit = "1")
    protected static final long doCompareAscii0Order(/unused/ final Object receiver,
                                                    final NativeObject string1,
                                                    final NativeObject string2,
                                                    /unused/ final NativeObject orderValue,
                                                    /unused/ @Cached("asciiOrderOrNull(orderValue)") final NativeObject cachedAsciiOrder,
                                                    @Cached final TruffleString.CompareBytesNode stringCompareNode

    ) {
        return compareAscii0Order(string1, string2, stringCompareNode) + 2L;
    }
}
```


Code Writing: Workflow and Discussion

- **Goals:** Define intended behavior, prototype new changes
- **Tools**
 - OSVM: Squeak, VS Code
 - TSVM: Squeak, IntelliJ IDEA
- **Frameworks**
 - OSVM: Low-level Slang, mostly **procedural**
 - TSVM: Higher-level Java, **declarative** Truffle framework
- **Discussion**
 - Squeak strong for **prototyping**
 - IntelliJ supports type-based **code completion** and **AI suggestions**

💡 OSVM: **Full type analysis** for code completion¹

💡 OSVM: **AI-based** code suggestions

Code Writing

Building

Code Browsing

Code Writing: Workflow and Discussion



- **Goals:** Define intended behavior, prototype new changes
- **Used tools**
 - Code browsing tools (Squeak browser, IntelliJ IDEA, ...)
 - **Prototyping:**
 - **OSVM:** Workspace, inspectors, debugger, ...
 - **TSVM:** JShell Console, Scratch files – still require building the project
 - 💡 First-class support for programming at runtime?
 - **Code completion:**
 - **TSVM:** IntelliJ with context from static type system; LLM support
 - **OSVM:** only lexical suggestions plus heuristic type analysis (third-party tools)
 - 💡 Real type analysis (e.g., LiveTyping); add LLM support
 - **Language and frameworks:**
 - **OSVM:** Low-level Slang, mostly procedural (for JIT slightly higher-level)
 - Data structures: OOPs, memory, assembler instructions
 - Harder to write, poorer code quality
 - 💡 Improve transpiler support; introduce higher-level abstractions?
 - 💡 Higher-level abstractions?
 - **TSVM:** High-level Java, declarative Truffle framework
 - Objects: TruffleNodes, Truffle Objects,
 - Better framework support
 - Possible restrictions and leaky abstractions by declarative framework

```
method := (Integer >> #benchFib) copy.  
method initialPC: "41"  
method at: 41 put: 217.  
Context runSimulated:  
    [method valueWithReceiver: 3 arguments: #()] 5
```

```
replInstSize := objectMemory fixedFieldsOf: repl format: replFmt length: totalLength.  
(replStart >= 1 and: [stop - start + replStart + replInstSize <= totalLength]) ifFalse:  
    [^self primitiveFailFor: PrimErrBadIndex].  
  
"Array formats (without byteSize bits, if bytes array) must be the same"  
arrayFmt < objectMemory firstByteFormat  
ifTrue: [arrayFmt = replFmt ifFalse:  
    [^self primitiveFailFor: PrimErrInappropriate]]  
ifFalse: [(arrayFmt bitAnd: objectMemory byteFormatMask) = (replFmt bitAnd:  
    objectMemory byteFormatMask) ifFalse:  
    [^self primitiveFailFor: PrimErrInappropriate]].  
  
srcIndex := replStart + replInstSize - 1.  
"- 1 for 0-based access"  
  
arrayFmt <= objectMemory lastPointerFormat  
ifTrue:  
    [start + arrayInstSize - 1 to: stop + arrayInstSize - 1 do:  
        [i |  
            objectMemory storePointer: i ofObject: array withValue: (objectMemory  
                fetchPointer: srcIndex ofObject: repl).  
            srcIndex := srcIndex + 1]]
```

eem 1/10/2017 16:38 · Eliot Miranda · indexing primitives · only in change set CogPools-ISAs

Building: Workflow and Discussion

- **Goals:** Compile & run, fix build errors
 - **Tools**
 - OSVM: Slang, Clang
 - TSVM: Truffle, GraalVM, mx
 - **Discussion**
 - TSVM: More detailed **documentation**
 - OSVM: **Earlier verification** through live programming & faster compilation
 - TSVM: **Static analysis** and Truffle-specific validations at **build time**
 - TSVM slower compilation than OSVM
-
- 💡 TSVM: Run validations **before building** (i.e., IntelliJ Plugin)
 - 💡 OSVM: **Improve validation** of Slang code (i.e., generate/check C files interactively)

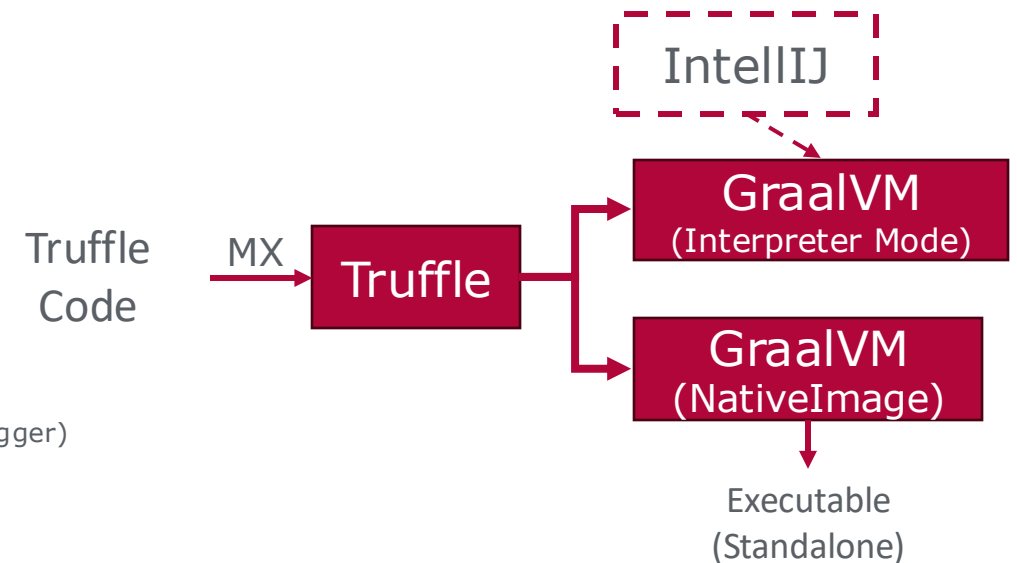
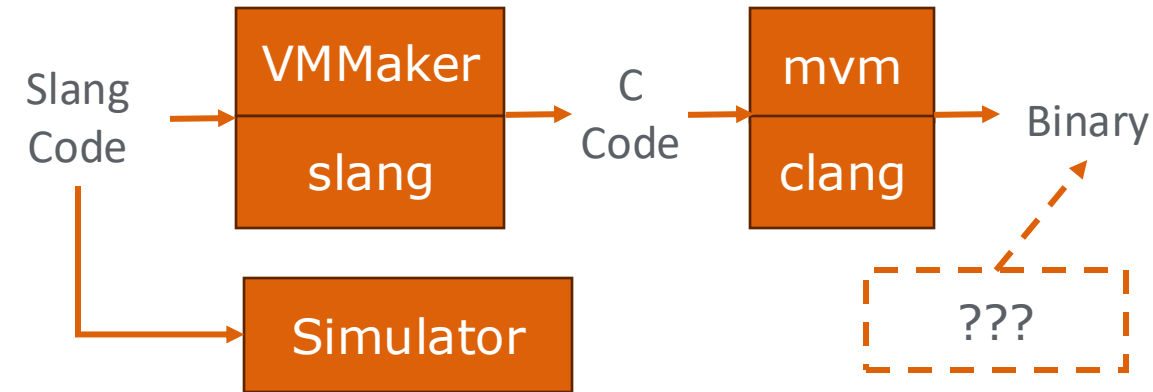
Code Writing

Building

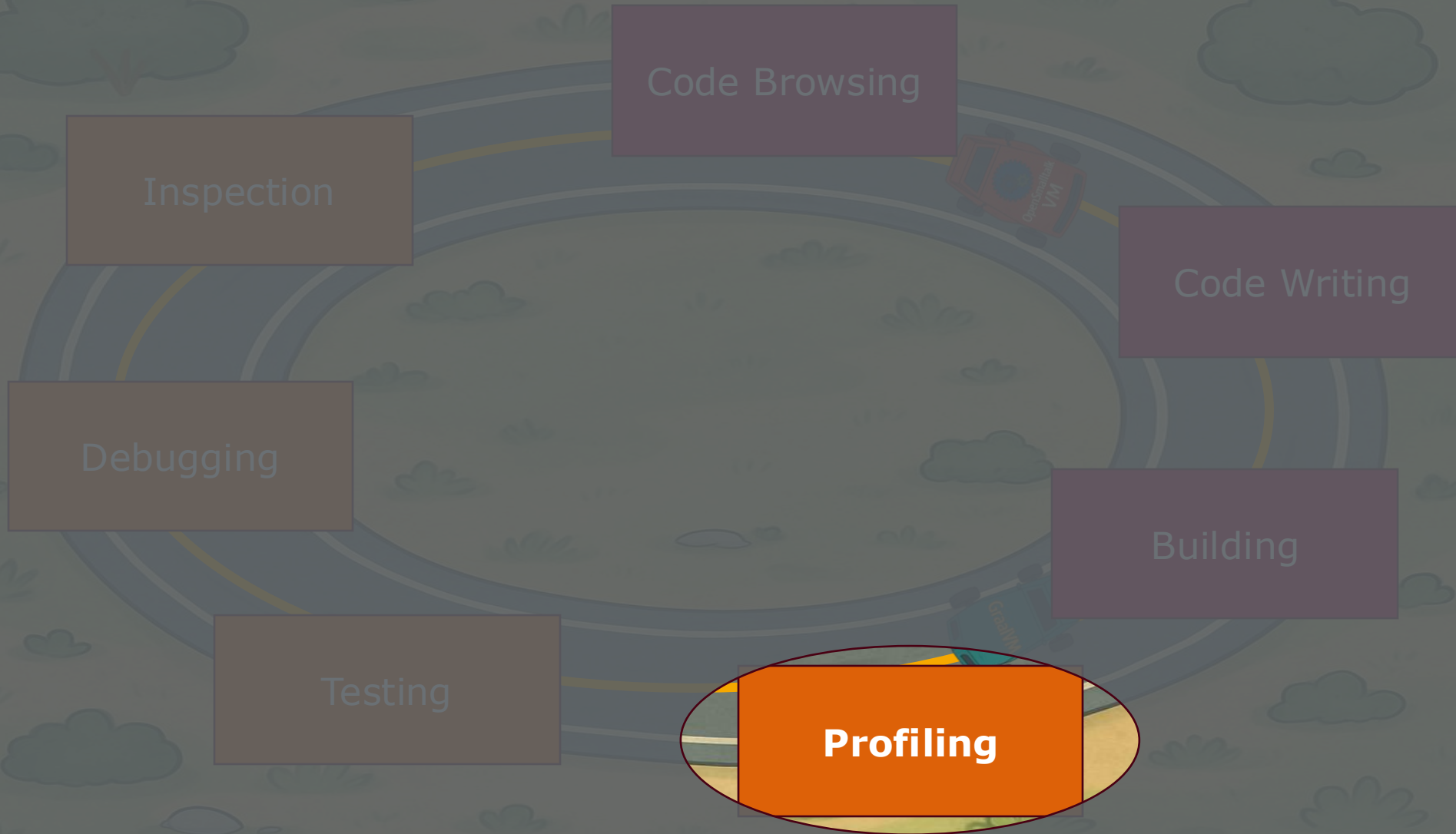
Building: Workflow and Discussion



- **Goals:** Compile & run
- **Used tools**
 - **TSVM:** mx (for Truffle), GraalVM (execution)
 - **OSVM:** VMMakerTool (for Slang), mvm (for Clang), Squeak (execution)
- **Discussion**
 - **Documentation:**
 - TSVM: More extensive instructions
 - OSVM: Build chain is easier to explore/change (transpilation)
 - Both occasionally contain outdated contents -->💡 Regular revisits?
 - 💡 OSVM: Improve self-explanation and default configuration of VMMakerTool
 - **Integration:**
 - **TSVM:** Editor annotations, CLI façade
 - **OSVM:** Squeak browsing tools, GUI for transpilation, CLI façade for compilation
 - 💡 First-class in-image support for compilation process
 - **Errors:**
 - TSVM:
 - Static analysis and Truffle-specific validations at build time
 - Interactive feedback for syntax errors and linter
 - Integration: Editor annotations and CLI output
 - 💡 Run validations before building (via IntelliJ plugin)?
 - OSVM:
 - Earlier verification through live programming (do-its, programming in the debugger)
 - Shorter edit-run cycles due to small granularity of methods
 - Slang issues at transpilation time
 - Integration: Simulation and transpilation in Squeak;



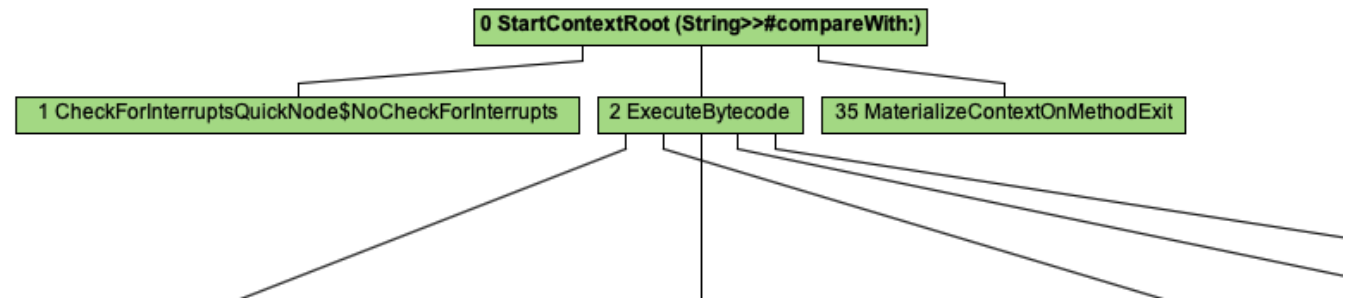
Profiling: Workflow and Discussion



Profiling: Workflow



- **Goals:** Gather, analyze and monitor performance data for later optimization
- **Used Tools**
 - **Benchmarks:** AWFY, Bouncing Atoms
 - **Profiling**
 - TSVM: **CPU-level**-time-profiling, Specialization-profiling
 - Both: **Message-level**-time-profiling, **Branch-level**-profiling
 - TSVM: **Compilation Artifact** Visualization (IGV, Performance Report)



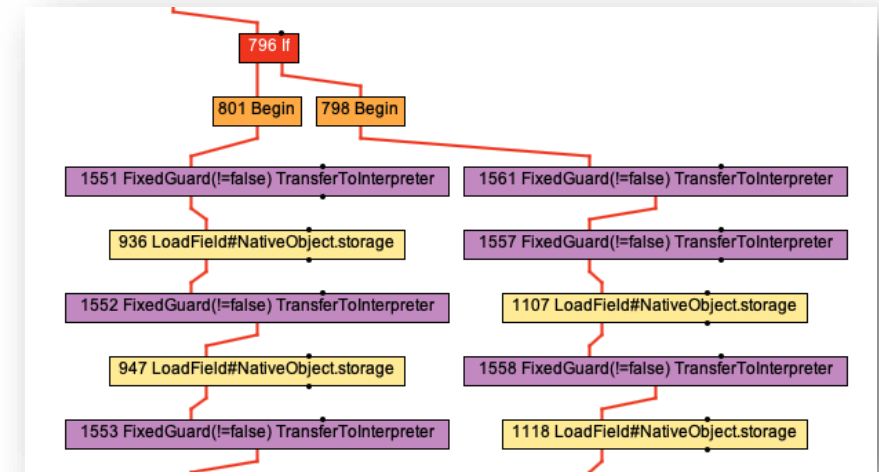
Profiling: Discussion

- **TSVM**

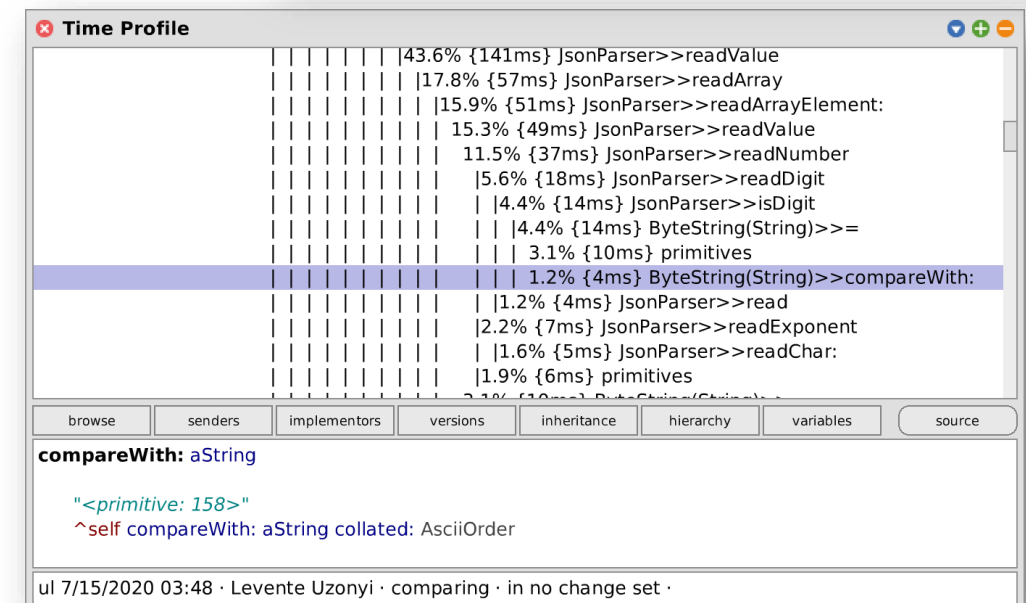
- Potential **information overload**
- Hard to detect **differences** between VM versions

- **OSVM**

- **Unstructured** text-based output & **duplicate** subtrees



- 💡 TSVM: **Compare** Compilation Logs 🏗️
- 💡 OSVM: **Visual Graph** of Scorch Control Flow
- 💡 OSVM: **Flamegraph**
- 💡 OSVM: **Code size** inspection



Testing: Workflow and Discussion

- **Goals:** **Verification** of VM behavior
- **Tools**
 - OSVM: **Manual** experiments, automated tests
 - TSVM: **Automated** tests
- **Discussion**
 - Same automated tests
 - Manual testing **accelerates prototyping** but needs **repeated test setup**

💡 OSVM: **Bytecode Editor** 🏗️

Testing

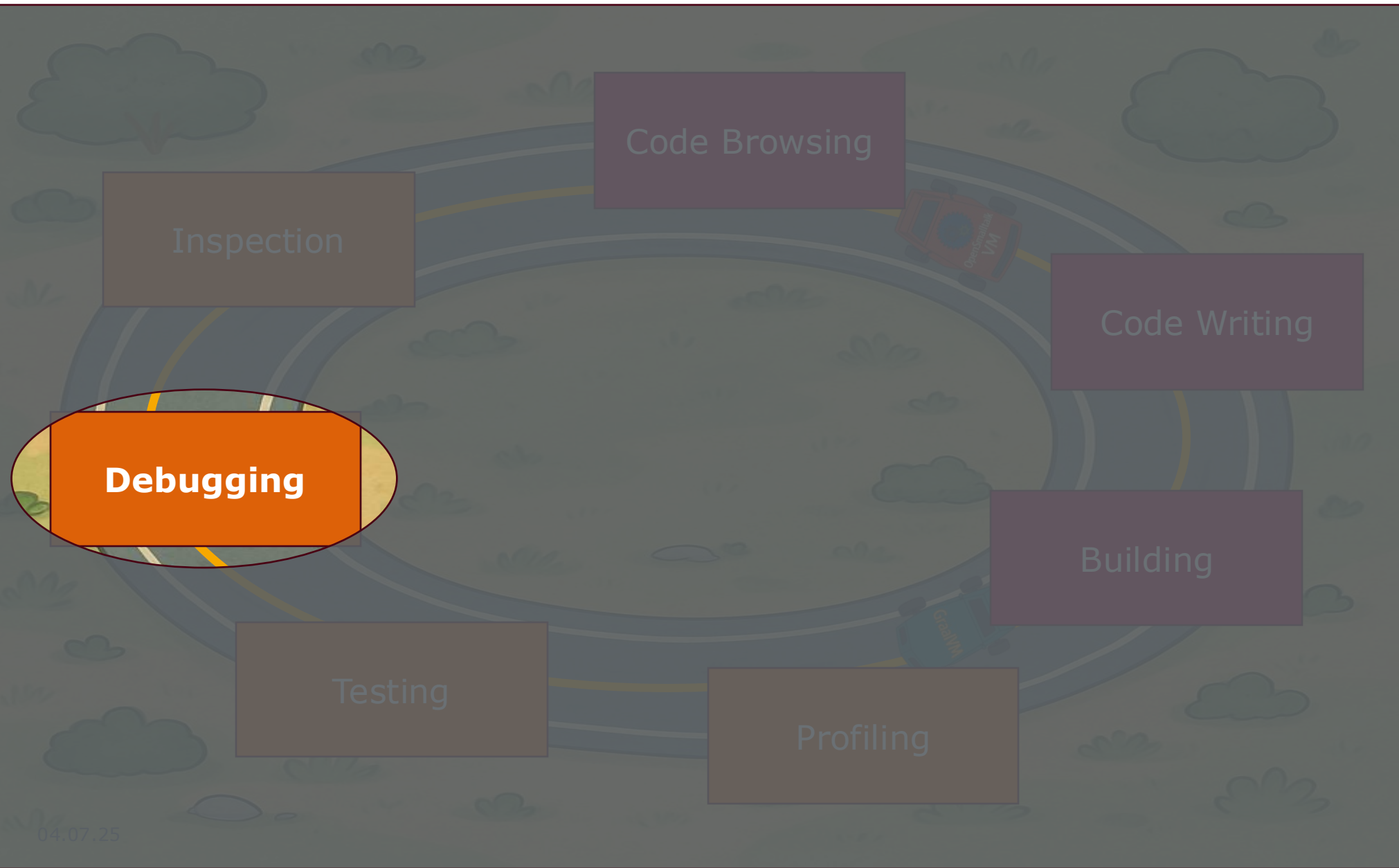
```
Object testString158 {experiments} {Kernel-Objects}
browse senders implement versions inheritance hierarchy variables assembler

self
  importMethodClass: #Object;
  selector: #testString158.
  self literal: 'abc'; literal: 'def'.

self pushLiteralVariableAtIndex: 0.
self pushLiteralVariableAtIndex: 1.
self callInlinePrimitive: 158.
self methodReturnTop.
```

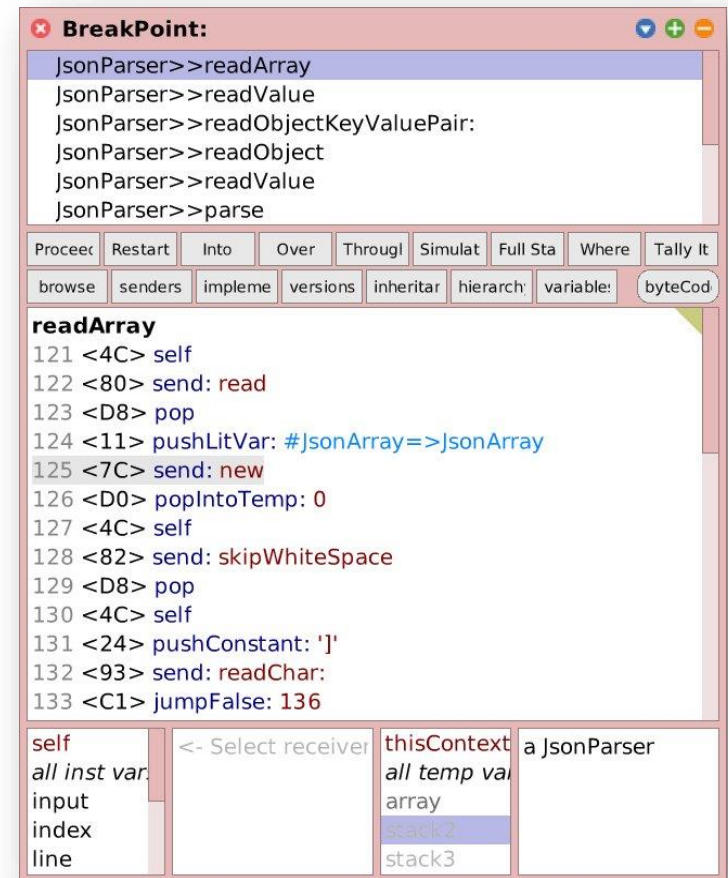
no timeStamp · unknown author · as yet unclassified · in no change set ·

Debugging: Workflow and Discussion



Debugging: Workflow

- **Goals: Explore code** interactively, by example
 - Identify + fix **defects**
 - **Understand** behavior
- **Used Tools**
 - **Debug setup: Scripts** to start image
 - OSVM: Nested **simulation** GUI + REPL
 - **Symbolic debugger:** VM-specific debugger
 - OSVM: + **Bytecode-level** debugger
 - **Breakpoints:**
 - TSVM: Line-based
 - OSVM: Statement-based + simulator-based **events**



single step
click step
break on execute method
set break pc...-ve to disable or remove
set break count...
set break selector...
set break block in coInterpreter...
set break block in cogit...
set cogit break method...

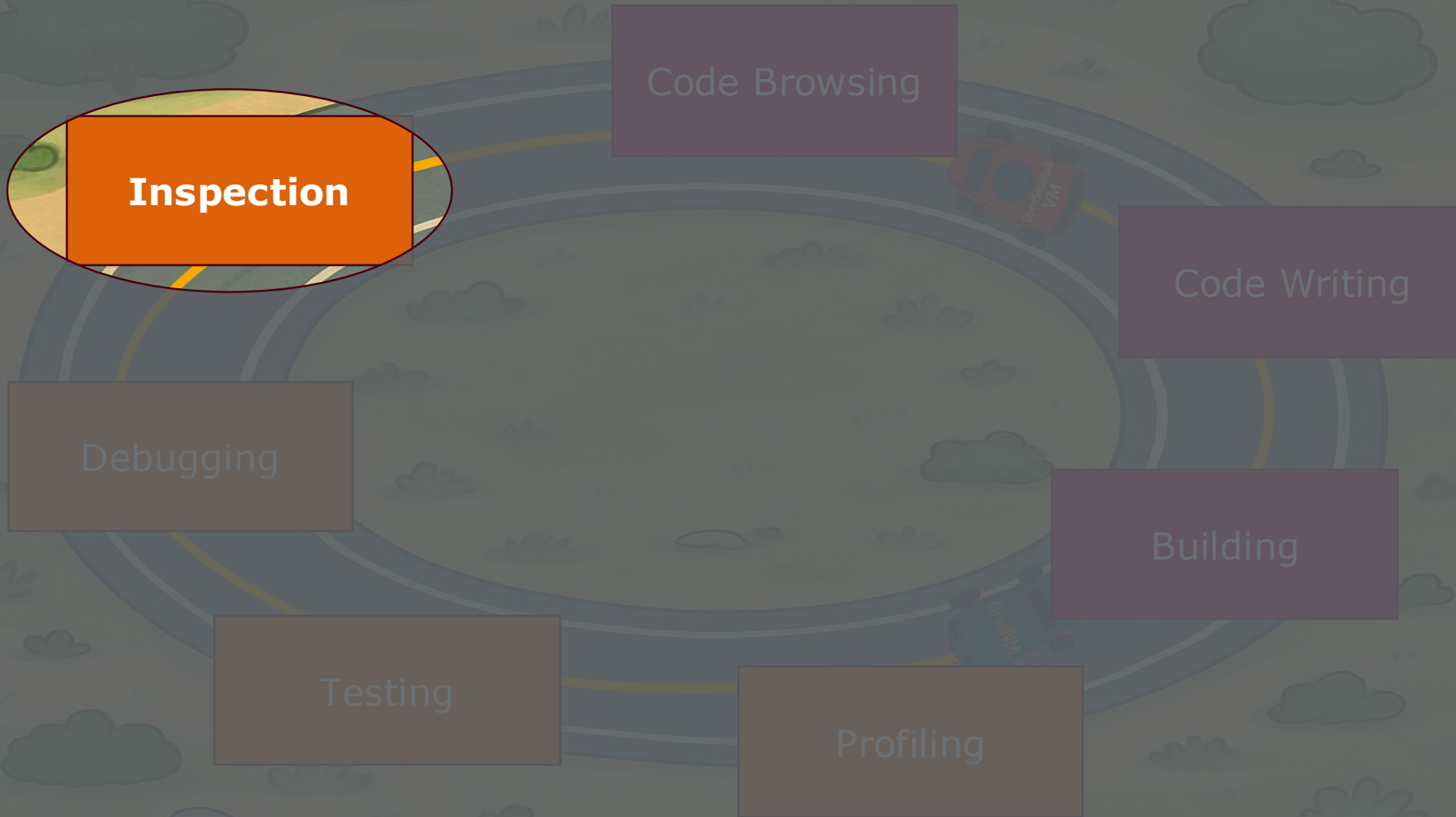
Debugging: Discussion



- **Slow** image startup/compilation process
- **TSVM: Spatial distance** between host debugger + Squeak image
- **OSVM:**
 - **Unhandled errors** in REPL expressions **interrupt** the workflow
 - **Modal REPL** prevents multitasking
 - Simulator GUI offers **non-interactive performance**
 - Limited debugging support for **simulated call stacks**

- 💡 TSVM: Improve **hot-reloading** support?
- 💡 OSVM: **Compress** image
- 💡 TSVM: Better **integration** of Squeak artifacts into **IntelliJ IDE**²
- 💡 OSVM: Add **syntax highlighting** and **checking** to REPL
- 💡 OSVM: Make REPL **non-modal** 🏗️

Inspection: Workflow and Discussion



Inspection: Workflow

```
squeak> 16r872020: a(n) JSONObject
          16r11 =2 (16r2)    16r872038 an Array
16r872038: a(n) Array
          16rA0CE08 nil      16rA0CE08 nil      16r872260 an Association #level -> 16r00
          16rA0CE08 nil
```

- **Goals:** **Explore** runtime artifacts, identify **defects**
- **Tools**
 - Object **inspection tools:** Squeak inspector, explorer
 - TSVM: IntelliJ debugger
 - OSVM: Custom inspectors, **Manual logging** of pointers (text-based)
 - **Disassemblers:** Squeak **bytecode** disassembler + VM-specific **machine code** disassembler
 - TSVM: **Graph-based** node visualization

```
Transcript
C5450
  objhdr: 8000000A000035
  nArgs: 0 type: 5 CMMMethod
  blksize: 1E0
  method: 1EEC4D0 in Integer
  mthdr: 8000000000000021
  selctr: BFD968 #benchFib
  blkentry: 0
  stackCheckOffset: 78/C54C8
  cmRefersToYoung: no cmHasMovableLiteral: no cmIsFullBlock: no
  counters:
000c5480: xorq %rdx, %rdx : 48 31 D2
000c5483: call .-0xC4BE0 (0x8a8=ceMethodAbort0Args) : E8 20 B4 F3 FF
entry:
000c5488: movq %rdx, %rax : 48 89 D0
000c548b: andq $0x7, %rax : 48 83 E0 07
000c548f: jnz .+0x9 (.+004A) : 75 09
000c5491: movq (%rdx), %rax : 48 8B 02
000c5494: andq $0x3ffff, %rax : 48 25 FF FF 3F 00
000c549a: cmpq %rcx, %rax : 48 39 C8
000c549d: jnz .-0x1C (.+0033) : 75 E4
```

```
send input to stdin
toggle transcript
deliver interrupt
clone VM
print head frame
short print head frame & callers
long print head frame & callers
print frame...
inspect frame...
print call stack
print stack call stack
print stack call stack of...
print call stack of...
print call stack of frame...
print all stacks
write back local ptrs
write back mc ptrs
print rump C stack
print registers
print register map
disassemble method/trampoline...
disassemble method/trampoline at pc
disassemble ext head frame method
print oop...
inspect oop...
long print oop...
print context...
symbolic method...
inspect object memory
run leak checker
inspect cointerpreter
inspect cogit
inspect method zone
inspect head frame
inspect processor
print cog methods
print cog methods with prim...
print cog methods with selector...
print cog methods with method...
print cog method for...
print cog method header for...
print trampoline table
print prim trace log
report recent instructions
print registers each instruction
print instructions each instruction
print bytecode each bytecode
print frame each bytecode
single step
click step
break on execute method
set break pc...-ve to disable or remove
set break count...
set break selector...
set break block in colnterpreter...
set break block in cogit...
set cogit break method...
```

Inspection: Discussion



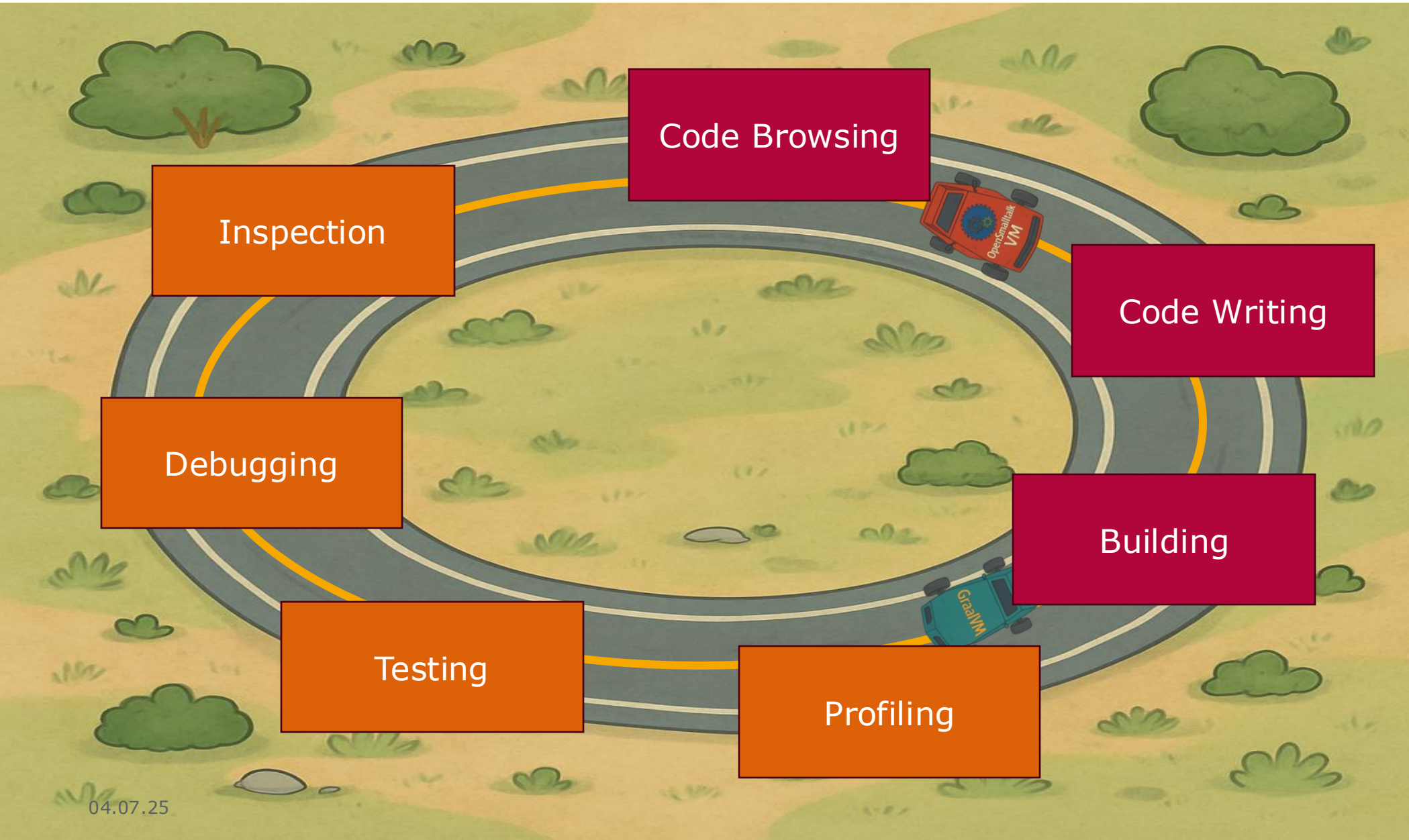
- Disassembled output **hard to read** and **explore**
- **TSVM:** IntelliJ inspector lacks **simple extensions** for Truffle objects
- **OSVM:** pointer logging is text-based — **no interactivity, manipulation,** or **liveness**

💡 **OSVM:** Improve **syntax support** for disassembler:
alternative DSL 🏗️, syntax highlighting 🏗️, tooltips, decompiler?

💡 **TSVM:** **Customizable object representations** in inspector

💡 **OSVM:** **Proxy inspectors** on objects from simulated image 🏗️

Main Learnings



1. Tooling focuses on **performance/profiling** (**TSVM**) vs. **debugging/inspection** (**OSVM**)
2. **Extensibility** of tooling: Better in **OSVM**
 - Rich visuals over raw text-based representation
3. Flexibility (**OSVM**) vs. **framework**-bound (**TSVM**)
4. **Static** (**TSVM**) vs. **dynamic** (**OSVM**) typing
5. **Conceptual shift** between two languages/tool sets
 - Windows & methods (**OSVM**) vs. tabs & files (**TSVM**)
6. **Documentation, discoverability, and integration** of different tools
 - **OSVM**: Focus on self-explaining, separate docs tend to be forgotten/outdated
 - **TSVM**: Focus on separate docs, more up-to-date

- **OSVM**
 - Further advance **proxy inspector system**
 - Fix bugs for **Sista/Scorch MVP**
- **TSVM**
 - Finish and validate **Diff Tool for Truffle Compilation Traces**
 - Replace **Byte Array** representation with **Truffle String** too

**Join us at our
end presentation
in two weeks!**

Which other notable pain points did you experience when developing/exploring VMs?



What novel tools would you like to use in your VM development workflow?